Diseño, carga y análisis de una base de datos relacional de streamers

Jefry Sneider Díaz Florez
Facultad de Ingeniería
jsdiazf@academia.usbbog.edu.co

Angel David Figueredo Reyes
Facultad de Ingeniería
adfigueredor@academia.usbbog.edu.co

Anna Sophia Rangel Ladino
Facultad de Ingeniería
asrangell@academia.usbbog.edu.co

***Resumen*—Este informe documenta la construcción de una base de datos relacional en MySQL a partir de un archivo plano con 999 streamers y su uso para responder ocho preguntas mediante SQL. Se argumenta que cada decisión de diseño, desde la selección de columnas hasta la arquitectura del programa de carga, se toma para que las consultas finales sean posibles y correctas. Se presentan el modelo entidad-relación normalizado, la carga en Node.js y cuatro consultas básicas y cuatro de análisis, cada una con su interpretación en el contexto del dataset.**

***Index Terms*—MySQL, normalización, modelo entidad-relación, SQL, Node.js, streamers.**

I. Introducción

En este laboratorio se construirá una base de datos relacional en MySQL a partir de `Streamers.csv`, un archivo plano de 999 streamers descritos por 17 columnas, y se usará para responder preguntas concretas mediante consultas SQL. El problema de fondo es que una tabla plana mezcla en cada fila hechos de naturaleza distinta (la identidad del streamer, su idioma, sus dos juegos más transmitidos y sus métricas de audiencia), y esa mezcla se paga después con redundancia y con consultas difíciles de escribir.

El informe seguirá un único argumento: cada decisión de diseño se justificará por lo que permite responder después. Primero se determinará qué columnas necesita el sistema (Sección II); esa selección definirá las entidades, que se normalizarán de forma secuencial (Sección III). El modelo resultante se traducirá a sentencias DDL y se alimentará con un programa de carga en Node.js cuya arquitectura se justificará (Sección IV). Sobre la base cargada se ejecutarán cuatro consultas básicas (Sección V) y cuatro consultas de análisis (Sección VI), y cada resultado se interpretará en el contexto del dataset. Por último, se discutirán las limitaciones y las conclusiones (Sección VII).

II. Análisis del dataset y selección de información

El primer paso es decidir qué información entra al sistema. El criterio adoptado no es la limpieza del archivo sino la necesidad: una columna se conserva solo si al menos una de las ocho consultas del laboratorio la requiere o si identifica a un streamer. Con este criterio, de las 17 columnas de `Streamers.csv` se conservan 8 (Tabla I) y se excluyen 9. Aquí B1–B4 denotan las consultas básicas y A1–A4 las de análisis.

Tabla I
Columnas conservadas y consultas que las requieren

Columna	Consultas
`NAME`	B1, B2, B3 (identifica al streamer)
`LANGUAGE`	B2, B3, B4, A2
`MOST_STREAMED_GAME`	B3, A1, A3
`2ND_MOST_STREAMED_GAME`	A1, A3
`TOTAL_FOLLOWERS`	B1, B2, A2, A3
`TOTAL_VIEWS`	B4, A3
`AVG_VIEWERS_PER_STREAM`	B3, A4
`ACTIVE_DAYS_PER_WEEK`	B1, A4

Las nueve columnas excluidas lo están por razones distintas. `RANK` es un orden definido por el origen del archivo cuyo criterio no está documentado: el streamer con más seguidores ocupa la posición 271, de modo que no coincide con el orden por seguidores, y ninguna consulta lo necesita porque los rankings pedidos se obtienen con `ORDER BY` sobre las métricas almacenadas. `TYPE` (`personality` o `esports`) es válida, pero ninguna de las ocho consultas la requiere; su utilidad real aparece al interpretar los resultados y se discute en la Sección VII. Las columnas `AVERAGE_STREAM_DURATION`, `FOLLOWERS_GAINED_PER_STREAM`, `AVG_GAMES_PER_STREAM`, `TOTAL_TIME_STREAMED` y `TOTAL_GAMES_STREAMED` son métricas de actividad correctas dentro del dataset, pero ajenas a las preguntas, que se centran en seguidores, vistas, espectadores, idioma, juegos y días activos. Finalmente, `MOST_ACTIVE_DAY` y `DAY_WITH_MOST_FOLLOWERS_GAINED` obligarían a modelar una entidad adicional (día de la semana) que ninguna consulta utiliza.

Antes de diseñar se inspeccionó el archivo y se encontraron cuatro rasgos que condicionan tanto el modelo como la carga. Primero, 77 streamers no tienen segundo juego y en 6 el segundo coincide con el primero, por lo que el segundo juego es un dato opcional. Segundo, el archivo contiene 213 cadenas distintas de juego, pero dos pares difieren solo en mayúsculas (`VALORANT`/`Valorant` y `TrackMania`/`Trackmania`), así que representan 211 juegos. Tercero, `NAME` no se repite en las 999 filas, ni siquiera ignorando mayúsculas, y por tanto identifica a cada streamer. Cuarto, existen valores en cero: 105 streamers con `TOTAL_VIEWS` igual a 0, 120 con `AVG_VIEWERS_PER_STREAM` igual a 0 y 4 con

ACTIVE_DAYS_PER_WEEK igual a 0, algo que será relevante al interpretar la consulta A4.

III. Modelo entidad-relación y normalización

Las ocho columnas conservadas describen tres conceptos: el streamer, el idioma en que transmite y los juegos que transmite. De ahí surgen tres entidades. Cada streamer transmite en exactamente un idioma y un idioma es compartido por muchos streamers (relación 1:N). Cada streamer declara uno o dos juegos, el principal y el secundario, y un mismo juego aparece en muchos streamers, ya sea como principal o como secundario (relación N:M). Como el modelo relacional no representa directamente una relación N:M, esta se resuelve con una tabla puente, `streamer_game`, cuyo atributo `position` (1 para el juego principal, 2 para el secundario) pertenece a la relación y no a ninguna de las dos entidades (Fig. 1).

La normalización parte de la tabla plana con las ocho columnas seleccionadas, que se escriben aquí con nombres abreviados:

$$R_0(\underline{name},\ language,\ game_1,\ game_2,\ followers,\ views,\ avg_viewers,\ active_days)$$

La clave candidata es `name`, como se verificó en la Sección II. Cada forma normal se presenta como un paso justificado [1]–[4].

III-A. Primera forma normal

R_0 viola la primera forma normal porque $game_1$ y $game_2$ son un grupo repetitivo: dos columnas con el mismo dominio y el mismo significado que solo se distinguen por su posición. El síntoma práctico es que preguntar si el juego principal más frecuente coincide con el más frecuente como secundario obliga a comparar dos columnas distintas, y que 77 filas tienen la segunda columna vacía. La solución es una fila por streamer y juego, con la posición como parte de la clave:

$$R_1(\underline{name,\ position},\ game,\ language,\ followers,\ views,\ avg_viewers,\ active_days)$$

Quien no tiene segundo juego simplemente no tiene esa fila, de modo que desaparecen los valores nulos del grupo repetitivo. El costo de este paso se hace visible de inmediato: R_1 tiene 1.915 filas en lugar de 999, y los atributos de 916 streamers aparecen duplicados en dos filas.

III-B. Segunda forma normal

En R_1 la clave es compuesta. El atributo $game$ depende de la clave completa (qué juego ocupa qué posición de qué streamer), pero $language$, $followers$, $views$, $avg_viewers$ y $active_days$ dependen solo de $name$, es decir, de una parte de la clave. Estas dependencias parciales explican la duplicación observada en el paso anterior y se eliminan separando los atributos del streamer:

$$R_{2a}(\underline{name},\ language,\ followers,\ views,\ avg_viewers,\ active_days)$$

$$R_{2b}(\underline{name,\ position},\ game)$$

III-C. Tercera forma normal

En R_{2a} se buscaron dependencias transitivas entre atributos no clave y no se encontró ninguna: el idioma no determina las métricas, y seguidores, vistas, espectadores promedio y días activos son medidas independientes entre sí. En R_{2b} el único atributo no clave es $game$. Por tanto, estrictamente, este paso no introduce cambios sobre el esquema anterior: ya cumple la tercera forma normal.

Aun así, el dataset presenta un dato redundante que la teoría no obliga a resolver pero que el diseño sí debe atender: los nombres de idioma y de juego se repiten como texto libre (por ejemplo, "English" aparece en 401 filas). La evidencia de que el texto libre es frágil está en los pares `VALORANT`/`Valorant` y `TrackMania`/`Trackmania` de la Sección II, que son el mismo juego escrito de dos maneras. Por eso se extraen dos catálogos, `language` y `game`, con nombre único y clave sustituta, y las tablas que los usan los referencian mediante claves foráneas. Así se evita la anomalía de actualización (renombrar un idioma exigiría modificar cientos de filas), se garantiza la integridad referencial y se obtienen las entidades necesarias para las consultas con `JOIN`. También se asigna a `streamer` una clave sustituta, manteniendo `name` como clave candidata única, para que las claves foráneas sean compactas.

III-D. Esquema final

El resultado es el modelo de la Fig. 1: `language`, `game`, `streamer` y `streamer_game`. La clave de `streamer_game` es (`streamer_id`, `position`) y no (`streamer_id`, `game_id`), porque así el propio esquema impide que un streamer tenga dos juegos principales o dos secundarios.

IV. Implementación del esquema y del programa de carga

IV-A. Esquema en MySQL

El modelo se tradujo a sentencias DDL sobre el motor InnoDB, que es el que soporta claves foráneas y transacciones [5]. El Listado 1 resume las cuatro tablas; el script completo, generado con MySQL Workbench, se entrega en el repositorio.

Listado 1. DDL resumido del modelo.

```
CREATE TABLE language (
  language_id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  PRIMARY KEY (language_id),
  UNIQUE KEY name_UNIQUE (name));

CREATE TABLE game (
  game_id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  PRIMARY KEY (game_id),
  UNIQUE KEY name_UNIQUE (name));

CREATE TABLE streamer (
  streamer_id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  total_followers INT NOT NULL,
  total_views INT NULL,
  avg_viewers_per_stream INT NULL,
  active_days_per_week DECIMAL(2,1) NOT NULL,
  language_id INT NOT NULL,
  PRIMARY KEY (streamer_id),
```

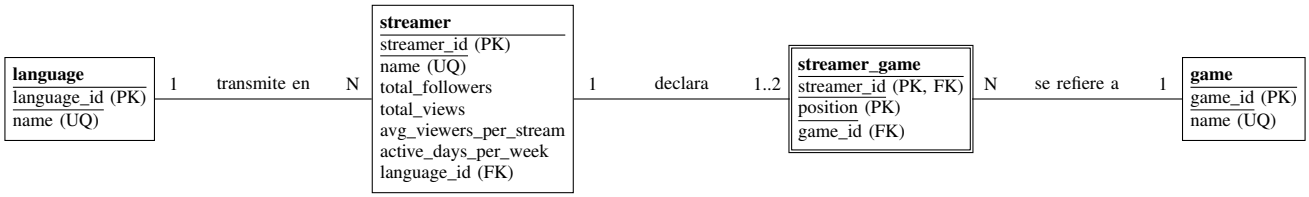


Fig. 1. Modelo entidad-relación normalizado. La tabla de doble borde resuelve la relación N:M entre streamer y juego; `position` distingue el juego principal (1) del secundario (2).

```
  UNIQUE KEY name_UNIQUE (name),
  CONSTRAINT fk_streamer_language
    FOREIGN KEY (language_id)
    REFERENCES language (language_id));

CREATE TABLE streamer_game (
  streamer_id INT NOT NULL,
  game_id INT NOT NULL,
  position TINYINT NOT NULL,
  PRIMARY KEY (streamer_id, position),
  CONSTRAINT fk_streamer_has_game_game1
    FOREIGN KEY (game_id)
    REFERENCES game (game_id),
  CONSTRAINT fk_streamer_has_game_streamer1
    FOREIGN KEY (streamer_id)
    REFERENCES streamer (streamer_id));
```

Los tipos de dato responden a lo observado en el archivo. Los seguidores (máximo de 19.000.000) y las vistas (máximo de 572.000.000) caben en `INT`, cuyo límite es 2.147.483.647 [5]. Los días activos se guardan como `DECIMAL(2,1)` porque son valores exactos de 0,0 a 7,0 con un decimal, y un tipo de punto flotante podría alterar comparaciones como `active_days_per_week >= 6`. Los nombres miden como máximo 57 caracteres, así que `VARCHAR(100)` es suficiente. El juego de caracteres `utf8mb3` basta porque el archivo no contiene caracteres de cuatro bytes. Las restricciones `NOT NULL` se aplican a los datos que identifican o filtran (nombre, seguidores, días activos, idioma), mientras que vistas y espectadores promedio admiten nulo. Finalmente, el índice `UNIQUE` sobre `name` en `game` usa el cotejo por defecto de MySQL, que no distingue mayúsculas [5]; por eso los pares `VALORANT`/`Valorant` y `TrackMania`/`Trackmania` se almacenan como un solo juego cada uno, lo que reduce los 213 textos del archivo a 211 juegos.

IV-B. Decisión arquitectónica del programa de carga

El programa de carga se implementó como un script independiente en Node.js (`cargar.js`) y no como un proyecto por capas de rutas, controladores y servicios. La razón es que esa arquitectura existe para atender solicitudes de clientes y separar responsabilidades que cambian a ritmos distintos, y la carga no tiene ninguna de las dos condiciones: es un proceso único, ejecutado por una persona desde la línea de comandos, sin interfaz de red. Agregar capas habría incorporado archivos e indirección sin ningún beneficio. La separación de responsabilidades se conservó a menor escala, dentro del script, mediante funciones pequeñas con una única tarea: leer el CSV, validar un valor numérico, validar un texto, resolver el identificador de un catálogo, cargar una fila y reportar. Si en el futuro la carga se expusiera como un servicio, estas funciones podrían moverse a una capa de servicios sin reescribirse.

La configuración no está escrita en el código: la ruta del archivo y el separador se reciben como argumentos de línea de comandos y los datos de conexión se leen de variables de entorno [6], con valores por defecto únicamente para un entorno local. El archivo `.gitignore` excluye `.env` y `node_modules` para que las credenciales no se publiquen. La lectura del CSV usa un flujo (*stream*) con la biblioteca `csv-parser` [8] y las inserciones usan consultas parametrizadas de `mysql2` [7], de modo que los valores nunca se concatenan en la sentencia SQL.

IV-C. Manejo de errores

El requisito es que la carga no falle en silencio. Se contemplaron los errores más previsibles en tres niveles. Antes de insertar, el programa comprueba que existan las columnas requeridas; si no, aborta e imprime las columnas encontradas, lo que delata un separador incorrecto. Para cada fila valida que los textos obligatorios no estén vacíos y que los valores numéricos sean convertibles, aceptando la coma decimal. Durante la inserción, las restricciones de la base de datos (único, no nulo, clave foránea y rango de tipo, este último bajo el modo SQL estricto) provocan una excepción que se captura.

Cada fila se carga dentro de su propia transacción [5]: la inserción del streamer, la de sus catálogos y la de sus uno o dos juegos entran juntas o no entran. Así un fallo al insertar el juego no deja un streamer sin juegos, y un error en una fila no cancela las demás. Cada fila descartada se registra con su línea, el nombre del streamer, el código del error y el detalle; el resumen se imprime en consola, se guarda en `errores_carga.json` y el proceso termina con código de salida distinto de cero si hubo algún error. Para los catálogos se usa `INSERT ... ON DUPLICATE KEY UPDATE` con `LAST_INSERT_ID`, que devuelve el identificador existente cuando el nombre ya estaba registrado [5]. Dos casos del dataset se tratan explícitamente: la celda vacía del segundo juego no genera fila, y tampoco la genera el segundo juego igual al primero, para respetar el modelo.

IV-D. Verificación de la carga

Para que la sesión de laboratorio no se pierda diagnosticando fallas de carga, la verificación se hizo antes: el programa imprime las filas leídas, insertadas y con error, y el conteo de registros por tabla. A partir del archivo

Falta la captura figuras/carga_conteo.png

Fig. 2. Conteo de registros por tabla tras la carga.

Tabla II
RESULTADO DE LA CONSULTA B1.

name	total_followers	active_days_per_week
xqc	12.000.000	6,3
thegrefg	11.900.000	6,3
sypherpk	6.940.000	6,1
amouranth	6.290.000	6,4
summit1g	6.270.000	6,7
elded	5.580.000	6,2
moistcr1tikal	5.400.000	6,2

se esperan 999 streamers, 19 idiomas, 211 juegos y 1.915 filas en `streamer_game` (999 juegos principales y 916 secundarios). Como control independiente, el CSV completo se importó a una tabla auxiliar, `datos`, que no forma parte del modelo, y se compararon sus conteos con los de las tablas del modelo (Fig. 2).

V. CONSULTAS BÁSICAS

Con la base cargada y verificada, las consultas se redactaron para responder una pregunta concreta; ninguna usa `SELECT *`. Las cuatro primeras cubren los tipos exigidos: filtro con dos condiciones, `JOIN` de dos entidades, `JOIN` de tres entidades y agregación con `GROUP BY`. Sirven además de base para las de análisis, porque muestran qué tipo de cuentas aparecen en el dataset.

V-A. B1: filtro con dos condiciones

¿ Qué streamers tienen más de cinco millones de seguidores y transmiten al menos seis días por semana?

```
SELECT name, total_followers, active_days_per_week
FROM streamer
WHERE total_followers > 5000000
  AND active_days_per_week >= 6
ORDER BY total_followers DESC;
```

Siete streamers cumplen ambas condiciones, el 0,7 % de los 999 del dataset (Tabla II). Los dos primeros, xqc (12,0 millones) y thegrefg (11,9 millones), están muy por encima del resto, que va de 5,4 a 6,9 millones. Todos transmiten entre 6,1 y 6,7 días por semana y ninguno llega a 7, de modo que la actividad alta es habitual en este grupo pero no diaria. Además, ocho de los diez streamers más seguidos (ver B2) no aparecen aquí y por tanto transmiten menos de seis días por semana. Es decir, tener muchos seguidores no implica transmitir casi todos los días, y la combinación de audiencia grande y frecuencia alta es poco común.

V-B. B2: JOIN de dos entidades

¿ Cuáles son los 10 streamers con más seguidores y en qué idioma transmiten?

```
SELECT s.name AS streamer, l.name AS idioma,
       s.total_followers
FROM streamer s
JOIN language l ON l.language_id = s.language_id
ORDER BY s.total_followers DESC
LIMIT 10;
```

Tabla III
RESULTADO DE LA CONSULTA B2.

streamer	idioma	total_followers
ninja	English	19.000.000
auronplay	Spanish	16.300.000
ibai	Spanish	15.600.000
rubius	Spanish	15.000.000
xqc	English	12.000.000
thegrefg	Spanish	11.900.000
juansguarnizo	Spanish	11.200.000
shroud	English	10.900.000
kaicenat	English	10.600.000
elmariana	Spanish	9.610.000

Tabla IV
RESULTADO DE LA CONSULTA B3.

streamer	idioma	juego_principal	avg_viewers
twitchrivals	English	League of Legends	481.615
ow_esports	English	Overwatch	400.009
gcxevent	English	Destiny 2	387.406
fortnite	English	Fortnite	352.166
rocketleague	English	Rocket League	349.645
riotgames	English	League of Legends	346.968
evo	English	Special Events	338.908
att	English	Counter-Strike	261.664
echo_esports	English	World of Warcraft	249.671
gaules	Portuguese	Counter-Strike	235.565

Ninja (inglés) encabeza con 19,0 millones, seguido por auronplay (16,3 millones), ibai (15,6 millones) y rubius (15,0 millones), los tres de habla hispana (Tabla III). Seis de los diez primeros transmiten en español y cuatro en inglés, aunque el español solo tiene 106 streamers en el dataset frente a 401 del inglés. Esto anticipa el resultado de A2: el español lidera el promedio de seguidores porque tiene muchos streamers grandes y no uno solo. Los valores 19,0 y 16,3 millones coinciden, además, con el mayor streamer de cada idioma en esa consulta. El décimo, elmariana, tiene 9,61 millones, por lo que el podio se concentra en dos idiomas.

V-C. B3: JOIN de tres o más entidades

¿ Qué juego principal e idioma tienen los 10 streamers con más espectadores promedio por transmisión?

```
SELECT s.name AS streamer, l.name AS idioma,
       g.name AS juego_principal,
       s.avg_viewers_per_stream
FROM streamer s
JOIN language l ON l.language_id = s.language_id
JOIN streamer_game sg
  ON sg.streamer_id = s.streamer_id
 AND sg.position = 1
JOIN game g ON g.game_id = sg.game_id
ORDER BY s.avg_viewers_per_stream DESC
LIMIT 10;
```

Nueve de los diez streamers con más espectadores promedio transmiten en inglés y el décimo, gaules, en portugués (Tabla IV). Encabezan twitchrivals (League of Legends, 481.615), ow_esports (Overwatch, 400.009) y gcxevent (Destiny 2, 387.406). Ninguno de los diez coincide con los de

	idioma	streamers	vistas_totales
▶	English	401	16101358904
	Spanish	106	2960614876
	Russian	115	2160223014
	Portuguese	82	2058089205
	French	72	2031652949
	German	60	1553331999
	Chinese	28	1228695000
	Japanese	79	682915080
	Polish	18	525813000
	Italian	13	154082130

Fig. 3. Resultado de la consulta B4.

B2, lo que indica que seguidores y audiencia promedio por transmisión miden cosas distintas. Por sus nombres, la mayoría parecen cuentas de torneos, eventos o empresas (twitchrivals, ow_esports, riotgames, rocketleague, fortnite, evo) y no creadores individuales, y una de las categorías es *Special Events*, que no es un juego. Esta lectura es una inferencia, y se contrastó con la columna `TYPE` del archivo original, excluida del modelo: ocho de los diez están etiquetados como `esports` (todos salvo gcxevent y att). League of Legends y Counter-Strike aparecen dos veces cada uno, lo que concuerda con el peso de League of Legends como segundo juego principal más común (A1).

V-D. B4: agregación con GROUP BY

¿ Cuántas vistas totales acumula cada idioma, considerando solo los idiomas con al menos 10 streamers?

```
SELECT l.name AS idioma, COUNT(*) AS streamers,
       SUM(s.total_views) AS vistas_totales
FROM streamer s
JOIN language l ON l.language_id = s.language_id
GROUP BY l.language_id, l.name
HAVING COUNT(*) >= 10
ORDER BY vistas_totales DESC;
```

La cláusula `HAVING` filtra grupos ya agregados, algo que `WHERE` no puede hacer porque se evalúa antes de agrupar [5].

El inglés suma 16.101 millones de vistas (Fig. 3), 5,4 veces las del español (2.960 millones) y cerca del 55 % del total de los diez idiomas listados. El orden por vistas no sigue el orden por cantidad de streamers: el chino, con 28 streamers, supera en vistas al japonés, con 79. Por streamer, el chino lidera con unos 43,9 millones de vistas, seguido del inglés con unos 40,2 millones, de modo que el peso del inglés se debe en buena parte a su tamaño y no solo a la audiencia por streamer.

VI. Consultas de análisis

Las cuatro preguntas asignadas al grupo de `Streamers.csv` se responden con el mismo modelo. En cada caso se responde primero la pregunta y después se matiza con las limitaciones de los datos.

Result Grid | Filter Rows: | Export:

	juego	como_principal	como_secundario
▶	Just Chatting	257	155
	League of Legends	84	36
	Grand Theft Auto V	74	40
	VALORANT	63	51
	Casino	36	1
	Dota 2	35	8
	Fortnite	35	39
	Counter-Strike	31	39
	Minecraft	24	42
	Virtual Casino	23	14

Fig. 4. Resultado de A1, ordenado por juego principal.

Tabla V
Resultado de A1b, ordenado por juego secundario.

juego	como_principal	como_secundario
Just Chatting	257	155
VALORANT	63	51
Minecraft	24	42
Grand Theft Auto V	74	40
Counter-Strike	31	39
Fortnite	35	39
Variety	4	37
League of Legends	84	36
Apex Legends	21	21
PUBG: BATTLEGROUNDS	12	18

VI-A. A1: juego principal más común

¿ Qué juego es el principal de más streamers? ¿ Coincide con el más frecuente como secundario?

Una sola consulta cuenta cada juego en ambas posiciones, aprovechando que en MySQL una comparación devuelve 1 o 0 y por tanto `SUM(sg.position = 1)` cuenta las filas que la cumplen [5]. Se ejecuta dos veces, ordenando por cada rol.

```
SELECT g.name AS juego,
       SUM(sg.position = 1) AS como_principal,
       SUM(sg.position = 2) AS como_secundario
FROM streamer_game sg
JOIN game g ON g.game_id = sg.game_id
GROUP BY g.game_id, g.name
ORDER BY como_principal DESC, g.name
LIMIT 10;
```

La segunda ejecución (A1b) es idéntica salvo por `ORDER BY como_secundario DESC, g.name`.

Just Chatting es el juego principal de 257 de los 999 streamers (25,7 %) y sí coincide con el más frecuente como secundario (155), como muestran la Fig. 4 y la Tabla V. El resto del ranking cambia: League of Legends es segundo como principal (84) pero baja al octavo lugar como secundario (36), donde lo preceden VALORANT (51) y Minecraft (42). Casino es principal de 36 streamers y secundario de solo 1, por lo que es un juego de nicho, mientras que Variety es principal de 4 y secundario de 37, es decir, funciona como complemento. En conjunto, el juego dominante lo es en ambos roles, pero los demás se especializan en uno de los dos.

idioma	streamers	promedio_seguidores	mayor_streamer	pct_del_mayor
Spanish	106	1776154	16300000	8.7
English	401	1133737	19000000	4.2
Turkish	7	1094714	2930000	38.2
Portuguese	82	775177	7110000	11.2
French	72	766026	5030000	9.1
Italian	13	740931	1820000	18.9
German	60	735735	5340000	12.1
Polish	18	588500	1950000	18.4
Korean	2	459500	587000	63.9
Thai	5	455800	827000	36.3
Russian	115	442713	3950000	7.8
Czech	3	343333	700000	68.0
Japanese	79	272054	1420000	6.6
Cantonese	2	264500	280000	52.9
Chinese	28	232975	565000	8.7
Hungarian	1	229000	229000	100.0
Ukrainian	3	205967	332000	53.7
Arabic	1	160000	160000	100.0
Romanian	1	141000	141000	100.0

Fig. 5. Resultado de la consulta A2.

VI-B. A2: idioma con mayor promedio de seguidores

¿ Qué idioma tiene el mayor promedio de seguidores por streamer? ¿ Es también el de más streamers, o el liderazgo se sostiene con pocos streamers muy grandes?

```
SELECT l.name AS idioma,
       COUNT(*) AS streamers,
       ROUND(AVG(s.total_followers)) AS promedio_seguidores
     ,
       MAX(s.total_followers) AS mayor_streamer,
       ROUND(MAX(s.total_followers) / SUM(s.total_followers
     ) * 100, 1)
         AS pct_del_mayor
FROM streamer s
JOIN language l ON l.language_id = s.language_id
GROUP BY l.language_id, l.name
ORDER BY promedio_seguidores DESC;
```

La consulta incluye `pct_del_mayor`, el porcentaje de los seguidores del idioma que concentra su mayor streamer, precisamente para poder responder la segunda parte de la pregunta.

El español tiene el mayor promedio, con 1.776.154 seguidores por streamer, un 57 % más que el inglés (1.133.737) (Fig. 5). No es el idioma con más streamers: el inglés tiene 401, el ruso 115 y el español 106. Su liderazgo tampoco depende de pocos streamers muy grandes, pues el mayor de ellos concentra solo el 8,7 % de los seguidores del idioma; el promedio lo sostienen muchos streamers grandes. En cambio, el turco, tercero en promedio, tiene solo 7 streamers y uno concentra el 38,2 %, y los idiomas con 1 a 3 streamers (húngaro, árabe, rumano, coreano, cantónés, checo y ucraniano) tienen concentraciones de entre 52,9 % y 100 %. Sus promedios no son representativos y no deben compararse con los de los idiomas grandes.

VI-C. A3: juego principal de unos, secundario de otros

¿ Existen streamers cuyo juego principal es el secundario de otro? ¿ Qué relación hay entre ellos en seguidores o vistas totales?

La pregunta se divide en dos consultas. La primera cuenta los streamers cuyo juego principal es el secundario de al menos otro streamer, mediante un `JOIN` de `streamer_game` consigo misma; la condición <> asegura que el otro streamer sea distinto.

streamers_cuyo_principal_es_secundario_de_otro
934

Fig. 6. Resultado de la consulta A3a.

```
SELECT COUNT(DISTINCT p.streamer_id)
         AS streamers_cuyo_principal_es_secundario_de_otro
FROM streamer_game p
JOIN streamer_game s
  ON s.game_id = p.game_id
 AND s.position = 2
 AND s.streamer_id <> p.streamer_id
WHERE p.position = 1;
```

La segunda compara, para cada juego, los seguidores y las vistas promedio de quienes lo tienen como principal frente a quienes lo tienen como secundario, limitada a juegos que cumplen ambos roles.

```
SELECT g.name AS juego,
  SUM(sg.position = 1) AS n_principal,
  ROUND(AVG(CASE WHEN sg.position = 1
    THEN s.total_followers END)) AS
     seguidores_prom_principal,
  ROUND(AVG(CASE WHEN sg.position = 1
    THEN s.total_views END)) AS vistas_prom_principal,
  SUM(sg.position = 2) AS n_secundario,
  ROUND(AVG(CASE WHEN sg.position = 2
    THEN s.total_followers END)) AS
     seguidores_prom_secundario,
  ROUND(AVG(CASE WHEN sg.position = 2
    THEN s.total_views END)) AS vistas_prom_secundario
FROM streamer_game sg
JOIN streamer s ON s.streamer_id = sg.streamer_id
JOIN game g ON g.game_id = sg.game_id
GROUP BY g.game_id, g.name
HAVING n_principal > 0 AND n_secundario > 0
ORDER BY n_principal + n_secundario DESC
LIMIT 15;
```

Sí: 934 de los 999 streamers (93,5 %) tienen un juego principal que es el secundario de otro (Fig. 6). Casi todos los juegos populares ocupan ambos roles, de modo que lo informativo es la relación entre los dos grupos, y no hay un patrón consistente (Fig. 7). Sobre los 15 juegos comparados, el grupo “principal” tiene más seguidores y más vistas en cuatro (Fortnite, Minecraft, World of Warcraft y PUBG); el grupo “secundario” los tiene en seis (VALORANT, Dota 2, Apex Legends, Variety, Virtual Casino y Call of Duty: Warzone); el resultado es mixto en cuatro (Just Chatting, League of Legends, GTA V y Counter-Strike); y Casino no tiene datos comparables en vistas. Por tanto, ser principal o secundario no se asocia de forma clara con más audiencia. Las limitaciones son que algunos promedios se basan en pocos streamers (Dota 2 como secundario: 8; Casino: 1) y que Casino tiene vistas iguales a 0, probablemente por datos faltantes en el origen.

VI-D. A4: días activos y espectadores promedio

Agrupando a los streamers por rango de días activos por semana, ¿ qué rango tiene el mejor promedio de espectadores por transmisión? ¿ Más días activos implica necesariamente más audiencia promedio?

	juego	n_principal	seguidores_prom_principal	vistas_prom_principal	n_secundario	seguidores_prom_secundario	vistas_prom_secundario
▶	Just Chatting	257	1271079	27789772	155	1130277	29366621
	League of Legends	84	833169	54601548	36	1411014	41227172
	Grand Theft Auto V	74	815434	20003934	40	1070768	17732526
	VALORANT	63	675278	12447278	51	1012396	32850659
	Fortnite	35	2802486	62415273	39	1782544	46204613
	Counter-Strike	31	795600	35880581	39	884587	19166582
	Minecraft	24	2168417	36766210	42	2009564	28125576
	Dota 2	35	412643	36804345	8	882250	39113750
	Apex Legends	21	725648	21578381	21	810767	29076479
	Variety	4	269057	8627500	37	349895	15206749
	Casino	36	1125	0	1	12	0
	Virtual Casino	23	6961	1354	14	539293	8739412
	World of Warcraft	18	1306889	74177778	15	810067	54675000
	Call of Duty: Warz...	20	938560	19656218	12	1503333	48917500
	PUBG: BATTLEGR...	12	1749167	98942500	18	1670667	86653889

Fig. 7. Resultado de la consulta A3b para los 15 juegos con más apariciones combinadas.

	rango_dias	streamers	espectadores_prom_por_stream
▶	1) menos de 2 dias	152	30861
	2) de 2 a 3.9 dias	277	20489
	3) de 4 a 5.9 dias	355	18492
	4) de 6 a 7 dias	91	27505

Fig. 8. Resultado de A4 con filtro.

Tabla VI
RESULTADO DE A4B SIN FILTRO.

rango_dias	streamers	espectadores_prom	maximo
1) menos de 2 dias	222	21.767	400.009
2) de 2 a 3.9 dias	305	18.608	481.615
3) de 4 a 5.9 dias	373	17.600	213.849
4) de 6 a 7 dias	99	25.283	235.565

Los rangos se construyen con `CASE`. Como el dataset contiene ceros (Sección II), la consulta se ejecutó de dos formas para mostrar cuánto cambia la conclusión: con filtro, que descarta los registros con 0 espectadores o 0 días activos, y sin filtro.

```
SELECT CASE
         WHEN active_days_per_week < 2 THEN '1) menos de 2
    dias'
         WHEN active_days_per_week < 4 THEN '2) de 2 a 3.9
    dias'
         WHEN active_days_per_week < 6 THEN '3) de 4 a 5.9
    dias'
         ELSE '4) de 6 a 7 dias'
       END AS rango_dias,
       COUNT(*) AS streamers,
       ROUND(AVG(avg_viewers_per_stream))
         AS espectadores_prom_por_stream
FROM streamer
WHERE avg_viewers_per_stream > 0
  AND active_days_per_week > 0
GROUP BY rango_dias
ORDER BY rango_dias;
```

La variante sin filtro (A4b) elimina la cláusula `WHERE` y agrega `MAX(avg_viewers_per_stream)` al `SELECT`.

El mejor rango depende de cómo se traten los ceros. Con filtro, el mejor es el de menos de 2 días, con 30.861 espectadores promedio; sin filtro, el de 6 a 7 días, con 25.283 (Fig. 8 y Tabla VI). El filtro excluye 124 streamers, 70 de ellos del primer rango, y por eso lo cambia todo. Lo que no cambia es la forma: el promedio baja hasta el rango de 4 a 5,9 días y vuelve a subir, de modo que más días activos no implica más audiencia, porque la relación tiene forma de "U" y no es creciente.

Esa forma debe leerse con cautela porque el promedio es sensible a valores extremos, con hasta 481.615 espectadores promedio y pocos streamers en los rangos extremos. Un cálculo complementario sobre el archivo, fuera de las consultas del laboratorio, lo muestra: con el mismo filtro, las medianas por rango crecen con los días activos (6.886, 6.936, 8.440 y 14.956 espectadores), y si se excluyen las cuentas etiquetadas `esports` el promedio del primer rango cae de 30.861 a 17.284. Es decir, el buen promedio de los que transmiten pocos días lo impulsan en buena medida unas pocas cuentas de eventos que ya aparecían en B3 (ow_esports en el primer rango y twitchrivals en el segundo), mientras que para el streamer típico sí parece haber una relación creciente.

VII. LIMITACIONES Y CONCLUSIONES

El hilo del informe fue que el diseño sirve a las preguntas: la selección de columnas fijó las entidades, la normalización resolvió el grupo repetitivo de los dos juegos con una tabla puente, y esa tabla es la que hizo posible A1 y A3, que habrían requerido comparar columnas o uniones manuales en la tabla plana. La extracción de catálogos, que la tercera forma normal no exigía, tuvo un efecto concreto: unificó variantes de un mismo juego escritas con distinta capitalización.

Los resultados tienen límites que deben tenerse presentes. Primero, el dataset mezcla creadores individuales con cuentas de torneos y empresas; la columna `TYPE`, excluida por no ser requerida por las ocho consultas, habría permitido separarlos, y una extensión natural del modelo sería incorporarla. Segundo, los valores en cero en vistas y espectadores obligan a decidir si se filtran, y esa decisión cambió la respuesta de A4. Tercero, los promedios de idiomas con pocos streamers (A2) o de juegos con pocos casos (A3) no son representativos. Cuarto, todas las relaciones halladas son asociaciones descriptivas y no establecen causalidad.

REFERENCIAS

[1] E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] E. F. Codd, “Further normalization of the data base relational model,” in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[3] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[5] Oracle Corporation, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] OpenJS Foundation, “Node.js documentation: process.” [Online]. Available: https://nodejs.org/docs/latest/api/process.html

[7] “mysql2: MySQL client for Node.js.” npm. [Online]. Available: https://www.npmjs.com/package/mysql2

[8] “csv-parser: streaming CSV parser.” npm. [Online]. Available: https://www.npmjs.com/package/csv-parser